

Multiple Linear Regression: Code from Scratch

Contents

1. Overview	2
2. The data	2
3. scikit-learn first	3
4. Our own class	3
4.1 fit	3
4.2 predict	3
5. Comparing the two	4
6. Solving without an explicit inverse	5
7. Summary	6
Sources	6
8. Key terms	6

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Regression problems ([Note 3](#)); Ordinary least squares (closed form) ([Note 51](#)); Simple linear regression ([Note 51](#)).
- **Leads to:** Polynomial regression ([Note 61](#)); Ridge regression ([Note 63](#)).
- **Compare with:** Gradient descent ([Note 57](#)); Moore-Penrose pseudo-inverse ([Note 613](#)); ANN for regression ([Note 1013](#)).

1. Overview

Key point: The normal equation fits in three lines of NumPy. Our own class gives the same coefficients and the same R^2 (0.44) as scikit-learn on the diabetes data.

The previous Note derived the normal equation:

$$\beta = (X^T X)^{-1} X^T y$$

This Note turns it into a class with `fit` and `predict`, like scikit-learn's `LinearRegression`, and checks that both give the same answer.

2. The data

Key point: 442 diabetes patients, 10 input columns, and a number to predict: how far the disease progressed one year later.

The example is scikit-learn's built-in **diabetes dataset**. Each row is a patient; the 10 inputs are age, sex, body mass index (bmi), average blood pressure (bp) and six blood measurements (s1 to s6). The target is a number measuring disease progression one year later.

Extra: The inputs come already scaled: each column is centred on 0 and shrunk so that its squares add up to 1 (scikit-learn docs, Diabetes dataset). The scaling explains why the values are small numbers such as 0.038. The target keeps its original scale, from 25 to 346.

We split the data 80/20: 353 patients to train, 89 to test.

Python: Loading and splitting.

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split

X, y = load_diabetes(return_X_y=True)    # (442, 10), (442,)
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.2, random_state=2)
```

3. scikit-learn first

Key point: scikit-learn's LinearRegression gives $R^2 = 0.44$ on the test patients, with 10 coefficients and an intercept of 151.88.

Python: The reference result.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

reg = LinearRegression().fit(X_train, y_train)
r2_score(y_test, reg.predict(X_test))    # 0.440
reg.intercept_                           # 151.88
```

An R^2 of 0.44 means the 10 measurements explain less than half the variation in disease progression: a useful but far from perfect model.

4. Our own class

Key point: fit adds a column of 1s and applies the normal equation; predict multiplies the inputs by the coefficients and adds the intercept.

4.1 fit

Key point: Three steps: add the 1s, compute β , split it into intercept and coefficients.

1. **Add a column of 1s** at the front of X , so the intercept is learned like any other coefficient. `np.insert(X, 0, 1, axis=1)` inserts the value 1 at position 0 along the columns. The training data goes from 353×10 to 353×11 .
2. **Apply the normal equation.** In NumPy, `@` is matrix multiplication, `.T` is the transpose and `np.linalg.inv` the inverse. The result has 11 numbers.
3. **Split the result:** the first number is the intercept β_0 , the other 10 are the coefficients.

4.2 predict

Key point: A prediction is the inputs times the coefficients plus the intercept: $\hat{y} = X\beta + \beta_0$.

For new data we do not need the column of 1s: we multiply the inputs by the 10 coefficients and add the intercept separately. For the test set that is an 89×10 matrix times a vector of 10, giving 89 predictions.

Python: Multiple linear regression from scratch.

```
import numpy as np
```

```

class MyMLR:
    def fit(self, X, y):
        # column of 1s at position 0, for the intercept
        Xb = np.insert(X, 0, 1, axis=1)
        # normal equation
        betas = np.linalg.inv(Xb.T @ Xb) @ Xb.T @ y
        self.intercept_ = betas[0]
        self.coef_ = betas[1:]
        return self

    def predict(self, X):
        return X @ self.coef_ + self.intercept_

mine = MyMLR().fit(X_train, y_train)
r2_score(y_test, mine.predict(X_test))    # 0.440

```

5. Comparing the two

Key point: All 11 numbers agree with scikit-learn to within about 0.000000000003.

Figure 1 (left) puts the 10 coefficients side by side: the two models are indistinguishable. The largest difference is about 3×10^{-12} , a rounding effect.

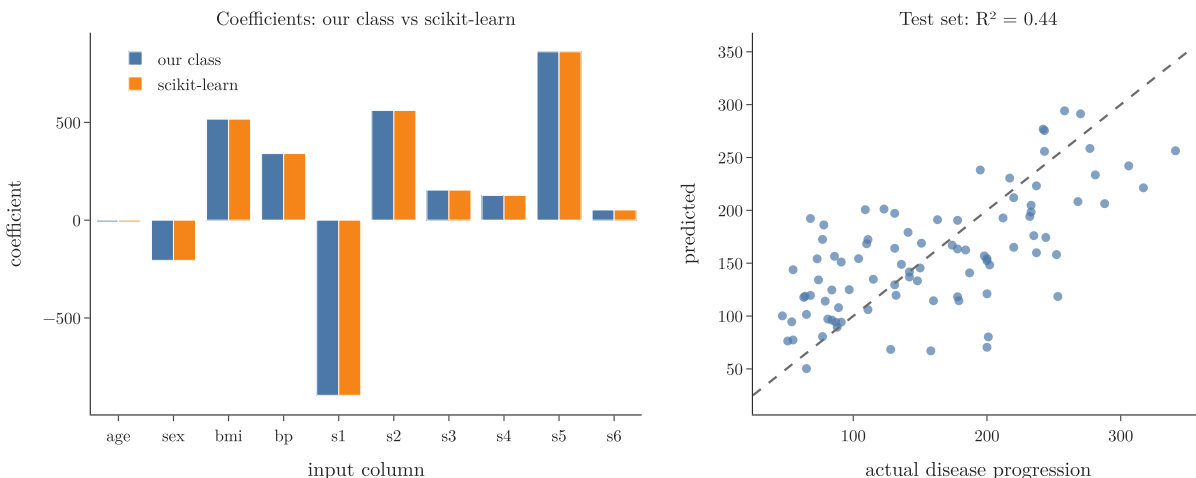


Figure 1: Coefficients of our class and scikit-learn; predicted against actual on the test set

The right panel plots the 89 test predictions against the true values. If the model were perfect, all points would lie on the dashed diagonal; the wide scatter is what an R^2 of 0.44 looks like.

The largest coefficients belong to s5 (+861), s1 (−896), s2 (+561) and bmi (+517). Because

the inputs are standardised, these can be compared with each other.

Extra: The large opposite coefficients of s1 and s2 are a typical sign of multicollinearity: s1 (total cholesterol) and s2 (LDL cholesterol) are strongly related, so the model can trade weight between them almost freely. Their individual sizes should not be over-interpreted. Regularisation, later, tames this effect. The Notebook checks all three points:

- **Strongly related:** correlation 0.895; variance inflation factors 56 and 37, against under 2 for age, sex, bmi and bp.
- **Trading weight:** over 200 bootstrap refits, the s1 and s2 coefficients swing far more than bmi's (standard deviation 464 and 369 against 77), in opposite directions (correlation -0.97).
- **Regularisation:** ridge regression with a small penalty (`alpha=0.1`) gives s1 -73 and s2 -81 .

6. Solving without an explicit inverse

Key point: In practice, libraries solve the least-squares problem from X itself instead of computing an inverse; the result is the same, and more accurate when columns are nearly copies of each other.

Computing $(X^T X)^{-1}$ explicitly is fine for a small example, but it can lose accuracy when the matrix is close to having no inverse. Two other NumPy functions give the same coefficients:

- `np.linalg.solve(Xb.T @ Xb, Xb.T @ y)` solves $X^T X \beta = X^T y$ directly.
- `np.linalg.lstsq(Xb, y)` finds the least-squares solution from X itself, without forming $X^T X$.

Both agree with scikit-learn to within 10^{-11} here (see the Notebook). The safer one is `lstsq`: when one column is almost a copy of another, `inv` and `solve` both lose accuracy, while `lstsq`, which never forms $X^T X$, stays accurate. scikit-learn's `LinearRegression` uses a least-squares solver of this second kind (scikit-learn docs, `LinearRegression`).

Extra: The Notebook's test uses a made-up design with known coefficients, where one column is a copy of another plus tiny noise. With noise of size 10^{-5} , the largest coefficient error is about 3×10^{-5} with `inv`, 9×10^{-5} with `solve` and 10^{-11} with `lstsq`. With noise of size 10^{-7} , `inv` and `solve` are off by about 0.08, `lstsq` by only 2×10^{-9} .

7. Summary

Step	Code
Add the intercept column	<code>np.insert(X, 0, 1, axis=1)</code>
Normal equation	<code>np.linalg.inv(Xb.T @ Xb) @ Xb.T @ y</code>
Intercept and coefficients	<code>betas[0], betas[1:]</code>
Predict	<code>X @ coef_ + intercept_</code>
Result on diabetes data	$R^2 = 0.44$, same as scikit-learn

- The normal equation needs only matrix products, a transpose and an inverse.
- Our class reproduces `LinearRegression` to about twelve decimal places.
- Real libraries avoid the explicit inverse; `lstsq`, which never forms $X^T X$, stays accurate when columns are nearly copies of each other.

Sources

- scikit-learn documentation, Toy datasets, Diabetes dataset. https://scikit-learn.org/stable/datasets/toy_dataset.html
- scikit-learn documentation, `sklearn.linear_model.LinearRegression`, Notes section (uses `scipy.linalg.lstsq`).

8. Key terms

Term	Meaning
Diabetes dataset	scikit-learn's built-in data of 442 patients, 10 standardised inputs, and disease progression one year later
<code>np.insert</code>	NumPy function that inserts values into an array at a given position
@ (matrix multiplication)	Python's operator for multiplying matrices and vectors
<code>np.linalg.inv</code>	NumPy function that computes the inverse of a square matrix
<code>np.linalg.lstsq</code>	NumPy function that finds the least-squares solution of a linear system